

FFJM2 vs. BFGS on the MGH Problems

1 Computational tests

In this section we evaluate the performance of `ffjm2` (Algorithm ??) and compare it against a classical sum-of-squares approach. We run two types of tests. In the first, we consider the MGH (Moré, Garbow, and Hillstom) sum-of-squares test problems [5], a benchmark of 34 classical problems; this part is important to us because it helps us settle on a good solver and Hessian approximation. In the second part, we solve a hydraulic problem in which we need to estimate a coefficient of a PDE called the Saint-Venant equation.

1.1 MGH problems

`ffjm2` minimizes $f(x) = \frac{1}{2}\|F(x)\|_2^2$ (Section 3, Eq. (16)) by minimizing, at each iteration, a local quartic model of the residuals in place of a line search. Following Algorithm 4.1 (Eq. (21)), the step s at iteration k solves

$$M_k(s) = \sigma\|s\|_2^2 + \frac{1}{2} \sum_{i=1}^m q_i(s)^2, \quad q_i(s) = f_i(x_k) + \nabla f_i(x_k)^\top s + \frac{1}{2} s^\top H_i s,$$

where f_i and ∇f_i are the i -th residual and its gradient at x_k (Section 3), H_i is the current quasi-Newton approximation to its Hessian (e.g. BFGS or SR1, Eqs. (19)/(20); four further variants are compared in Table 1 below), and $\sigma\|s\|_2^2$ is the Levenberg-Marquardt-style regularization of Algorithm 4.1 that damps the step in place of a line search. The candidate $x_k + s$ is checked for feasibility as in Step 3 (Eq. (24)); acceptance then uses the ratio ρ_k between the actual and the model-predicted reduction in f — a refinement of Algorithm 4.1’s simpler decrease test (Eq. (25)) — accepting whenever $\rho_k \geq \eta_1$ for a fixed threshold η_1 .

The implementation tested below also refines Algorithm 4.1’s regularization update (Eq. (30)) with an additional decay on acceptance. The first iteration starts at $\sigma = 1$. Each accepted step now divides σ by a fixed factor $\sigma_{grow} = 10$, with a floor $\sigma_{min} = 10^{-8}$, so σ keeps shrinking — unlike Algorithm 4.1’s plain carryover $\sigma_k \leftarrow \sigma$ on acceptance. Each rejected step, as in Eq. (30), multiplies σ by the same factor $\sigma_{grow} = 10$ and re-solves the subproblem at the same point, until a step is accepted or σ reaches a ceiling $\sigma_{max} = 10^{12}$ (not present in the base algorithm). Unlike a scheme that resets σ at the start of every iteration, this value always carries over from the previous one; this decaying carryover performed better in practice than restarting σ from scratch each time.

Every run reported below stops under one of three point-wise criteria, evaluated with the same default tolerances for `ffjm2` and for Optim.jl’s BFGS [4] (kept equal so the comparison is fair):

- **Gradient:** the gradient norm reaches $\|\nabla f\| \leq 10^{-8}$. This is a genuine first-order stationary point.
- **Step:** the last step has (numerically) zero length. The solver stalls and stops making progress.
- **Function:** the objective value does not change between two consecutive iterations. The iterate is stuck on a plateau.
- **Stalled:** FFJM2’s regularization parameter exceeded its ceiling (or its number of consecutive rejections exceeded its cap) before any step was accepted. Unlike the first three, this is *not* a converged outcome.

A run that exhausts the iteration budget without meeting any of the first three would be reported separately as *not converged*. This case does not appear in Tables 1 and 2 below, but it does in Table 3: FFJM2 (PSB) now stops under the **Stalled** criterion on MGH10, MGH16, and MGH31 (the same three problems that previously stopped under Step/Function — see the discussion after that table).

Table 1 compares the six ffjm2 Hessian-update models, each run independently on the 34 MGH least-squares problems [5]. Its “Converged (gradient)” column counts how many of the 34 problems stopped under the gradient criterion; the remaining columns are mean $\pm$ standard deviation over the 34 problems. Each model keeps its own set of matrices $H_i \approx \nabla^2 f_i(x_k)$, $i = 1, \dots, m$, updated at every accepted step from $s = x_{k+1} - x_k$ and $y_i = \nabla f_i(x_{k+1}) - \nabla f_i(x_k)$:

- **BFGS**: the classical rank-two secant update (Eq. (19)),

$$H_i \leftarrow H_i - \frac{H_i s s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i}.$$

- **SR1**: the symmetric rank-one update (Eq. (20)),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s)(y_i - H_i s)^T}{(y_i - H_i s)^T s}.$$

- **DW** (Dennis-Wolkowicz [1], Theorem 4.5(iii)): a Broyden-class update that adds a third, symmetrizing rank-one term to the BFGS pair,

$$H_i \leftarrow H_i - \frac{H_i s s^T H_i}{s^T H_i s} + \frac{y_i y_i^T}{s^T y_i} + (s^T y_i) w_i w_i^T, \quad w_i = \frac{y_i}{s^T y_i} - \frac{H_i s}{s^T H_i s}.$$

- **IS+BFGS** (interval-scaled BFGS [2]): BFGS with an extra self-scaling factor τ applied to the outdated part of H_i before the update,

$$H_i \leftarrow \tau \left(H_i - \frac{H_i s s^T H_i}{s^T H_i s} \right) + \frac{y_i y_i^T}{s^T y_i}, \quad \tau = \begin{cases} 1/\gamma & \text{if } \bar{\gamma}^{-1} \leq \gamma \leq \bar{\gamma} \\ 1 & \text{otherwise,} \end{cases} \quad \gamma = \frac{s^T H_i s}{s^T y_i},$$

with $\bar{\gamma} = 10$ (the default `gamma_bar`).

- **IS+DW** (interval-scaled DW [2]): the same interval scaling τ applied inside the DW update above,

$$H_i \leftarrow \tau \left(H_i - \frac{H_i s s^T H_i}{s^T H_i s} \right) + \frac{y_i y_i^T}{s^T y_i} + (s^T y_i) w_i w_i^T.$$

- **PSB** (Powell-Symmetric-Broyden [6]): a rank-two update that does not preserve positive definiteness (like SR1),

$$H_i \leftarrow H_i + \frac{(y_i - H_i s)s^T + s(y_i - H_i s)^T}{s^T s} - \frac{s^T(y_i - H_i s)}{(s^T s)^2} s s^T.$$

PSB and SR1 are the two best models: together they solve the most problems under the gradient criterion, and PSB in particular does so with by far the fewest iterations, evaluations, and execution time on average (Table 1). SR1 nonetheless certifies gradient convergence on one more problem than PSB (32/34 against 31/34). To see exactly where they disagree, Table 2 isolates the three problems on which at least one of the two methods does not stop under the gradient criterion.

Table 1: Comparison of the ffjm2 Hessian-update models on the 34 MGH problems.

Model	Converged (gradient)	Iterations	Func. evals	Grad. evals	Time (s)
BFGS	29/34	56.88 ± 188.25	99.97 ± 351.85	57.88 ± 188.25	2.071 ± 6.121
SR1	32/34	23.12 ± 54.84	44.32 ± 112.92	24.12 ± 54.84	0.934 ± 2.992
IS+BFGS	29/34	44.00 ± 170.39	88.15 ± 343.34	45.00 ± 170.39	1.590 ± 5.763
DW	29/34	56.12 ± 181.91	103.24 ± 351.45	57.12 ± 181.91	1.865 ± 5.942
IS+DW	29/34	49.41 ± 173.04	93.74 ± 344.17	50.41 ± 173.04	1.768 ± 5.923
PSB	31/34	15.68 ± 22.53	31.38 ± 53.44	16.68 ± 22.53	0.424 ± 0.919

Table 2: FFJM2 (PSB) vs. FFJM2 (SR1): per-problem comparison on the MGH problems.

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH10	FFJM2 (PSB)	4.40×10^1	5.75×10^{-5}	299	127	step
MGH10	FFJM2 (SR1)	4.40×10^1	5.50×10^{-2}	297	125	step
MGH16	FFJM2 (PSB)	4.29×10^4	9.26×10^{-5}	98	36	function
MGH16	FFJM2 (SR1)	4.29×10^4	6.26×10^{-5}	44	9	function
MGH31	FFJM2 (PSB)	1.53	2.53×10^{-8}	62	21	function
MGH31	FFJM2 (SR1)	1.53	4.39×10^{-9}	44	29	gradient

On MGH10 and MGH16, neither model reaches the gradient criterion: both stall under the step and function criteria, respectively, at essentially the same objective value, although FFJM2 with PSB reaches a smaller gradient norm on MGH10. MGH31 is the one case that favors SR1 over PSB. SR1 certifies a small gradient norm ($\|\nabla f\| \approx 4.4 \times 10^{-9}$), while PSB stops one iteration earlier under the function criterion, even though both reach practically the same objective value ($f \approx 1.5286$) and gradient norm. For this reason, PSB is the best choice between the two approaches when weighing cost against benefit.

FFJM2 (PSB) uses persistent decaying σ , a full Hessian reset on a bad ratio, and $-\nabla f + \text{Armijo}$ at $k = 0$ and as a safeguard. The following table compares it against BFGS with backtracking line search on the 34 MGH least-squares problems.

Table 3: FFJM2 (PSB) vs. BFGS (backtracking): per-problem comparison on the MGH problems.

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH01	FFJM2 (PSB)	2.42×10^{-22}	9.82×10^{-12}	12	8	gradient
MGH01	BFGS (backtracking)	1.98×10^{-22}	3.69×10^{-10}	53	40	gradient
MGH02	FFJM2 (PSB)	5.10×10^{-18}	3.84×10^{-9}	10	7	gradient
MGH02	BFGS (backtracking)	1.11×10^{-26}	2.20×10^{-12}	15	11	gradient
MGH03	FFJM2 (PSB)	1.58×10^{-11}	2.57×10^{-9}	31	20	gradient
MGH03	BFGS (backtracking)	5.05×10^{-30}	2.83×10^{-10}	233	174	gradient
MGH04	FFJM2 (PSB)	3.81×10^{-17}	8.73×10^{-9}	10	8	gradient
MGH04	BFGS (backtracking)	9.86×10^{-32}	4.44×10^{-10}	40	14	gradient
MGH05	FFJM2 (PSB)	1.31×10^{-16}	6.95×10^{-9}	10	8	gradient
MGH05	BFGS (backtracking)	2.50×10^{-22}	1.08×10^{-10}	22	19	gradient
MGH06	FFJM2 (PSB)	1.01×10^3	1.16×10^{-28}	11	2	gradient

continued on next page

Table 3: (continued)

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH06	BFGS (backtracking)	1.01×10^3	1.16×10^{-28}	10	2	gradient
MGH07	FFJM2 (PSB)	1.49×10^{-17}	6.03×10^{-9}	17	10	gradient
MGH07	BFGS (backtracking)	5.24×10^{-19}	5.03×10^{-9}	41	30	gradient
MGH08	FFJM2 (PSB)	4.11×10^{-3}	4.90×10^{-10}	12	9	gradient
MGH08	BFGS (backtracking)	4.11×10^{-3}	7.39×10^{-10}	34	28	gradient
MGH09	FFJM2 (PSB)	5.64×10^{-9}	7.55×10^{-11}	9	7	gradient
MGH09	BFGS (backtracking)	5.64×10^{-9}	1.55×10^{-9}	6	5	gradient
MGH10	FFJM2 (PSB)	4.40×10^1	1.20×10^{-5}	276	122	stalled
MGH10	BFGS (backtracking)	4.40×10^1	2.28	552	386	step
MGH11	FFJM2 (PSB)	5.96×10^{-21}	3.25×10^{-13}	26	17	gradient
MGH11	BFGS (backtracking)	2.27×10^{-20}	7.17×10^{-10}	50	42	gradient
MGH12	FFJM2 (PSB)	3.80×10^{-18}	3.81×10^{-10}	14	11	gradient
MGH12	BFGS (backtracking)	1.58×10^{-21}	3.20×10^{-11}	53	41	gradient
MGH13	FFJM2 (PSB)	5.16×10^{-12}	9.61×10^{-9}	12	9	gradient
MGH13	BFGS (backtracking)	1.57×10^{-14}	3.61×10^{-9}	55	45	gradient
MGH14	FFJM2 (PSB)	9.42×10^{-19}	8.23×10^{-10}	11	7	gradient
MGH14	BFGS (backtracking)	3.52×10^{-22}	5.46×10^{-10}	44	31	gradient
MGH15	FFJM2 (PSB)	1.54×10^{-4}	9.06×10^{-9}	38	22	gradient
MGH15	BFGS (backtracking)	1.54×10^{-4}	4.13×10^{-10}	39	33	gradient
MGH16	FFJM2 (PSB)	4.29×10^4	6.53×10^{-5}	91	37	stalled
MGH16	BFGS (backtracking)	4.29×10^4	2.83×10^{-5}	72	26	function
MGH17	FFJM2 (PSB)	2.73×10^{-5}	8.38×10^{-9}	24	12	gradient
MGH17	BFGS (backtracking)	2.73×10^{-5}	8.23×10^{-9}	90	59	gradient
MGH18	FFJM2 (PSB)	1.00×10^{-18}	1.65×10^{-11}	58	34	gradient
MGH18	BFGS (backtracking)	2.83×10^{-3}	1.28×10^{-8}	44	41	gradient
MGH19	FFJM2 (PSB)	4.38×10^{-2}	9.20×10^{-9}	115	59	gradient
MGH19	BFGS (backtracking)	1.37×10^{-1}	8.29×10^{-10}	147	113	gradient
MGH20	FFJM2 (PSB)	1.14×10^{-3}	1.84×10^{-9}	13	10	gradient
MGH20	BFGS (backtracking)	1.14×10^{-3}	7.07×10^{-9}	49	39	gradient
MGH21	FFJM2 (PSB)	2.42×10^{-21}	3.11×10^{-11}	12	8	gradient
MGH21	BFGS (backtracking)	4.79×10^{-18}	1.50×10^{-9}	228	150	gradient
MGH22	FFJM2 (PSB)	4.52×10^{-14}	2.80×10^{-10}	13	10	gradient
MGH22	BFGS (backtracking)	1.65×10^{-13}	1.56×10^{-8}	136	112	gradient
MGH23	FFJM2 (PSB)	1.12×10^{-5}	2.28×10^{-9}	13	10	gradient
MGH23	BFGS (backtracking)	1.12×10^{-5}	1.21×10^{-8}	87	68	gradient
MGH24	FFJM2 (PSB)	1.73×10^{-6}	1.74×10^{-9}	37	22	gradient
MGH24	BFGS (backtracking)	1.73×10^{-6}	6.29×10^{-9}	588	450	gradient
MGH25	FFJM2 (PSB)	7.40×10^{-32}	3.85×10^{-16}	13	6	gradient
MGH25	BFGS (backtracking)	6.84×10^{-26}	7.27×10^{-12}	23	16	gradient
MGH26	FFJM2 (PSB)	1.40×10^{-5}	3.00×10^{-9}	27	16	gradient

continued on next page

Table 3: (continued)

Problem	Method	f	$\ \nabla f\ $	Func. evals	Grad. evals	Criterion
MGH26	BFGS (backtracking)	1.40×10^{-5}	1.11×10^{-8}	36	34	gradient
MGH27	FFJM2 (PSB)	2.24×10^{-16}	1.94×10^{-9}	11	8	gradient
MGH27	BFGS (backtracking)	5.73×10^{-20}	3.69×10^{-9}	20	17	gradient
MGH28	FFJM2 (PSB)	2.22×10^{-15}	6.93×10^{-9}	10	8	gradient
MGH28	BFGS (backtracking)	2.53×10^{-16}	1.09×10^{-8}	26	18	gradient
MGH29	FFJM2 (PSB)	1.13×10^{-22}	1.91×10^{-11}	8	7	gradient
MGH29	BFGS (backtracking)	9.10×10^{-19}	1.56×10^{-9}	8	8	gradient
MGH30	FFJM2 (PSB)	7.64×10^{-27}	3.46×10^{-13}	10	7	gradient
MGH30	BFGS (backtracking)	1.97×10^{-19}	4.03×10^{-9}	46	23	gradient
MGH31	FFJM2 (PSB)	1.53	4.17×10^{-8}	49	18	stalled
MGH31	BFGS (backtracking)	1.34	4.48×10^{-9}	78	43	gradient
MGH32	FFJM2 (PSB)	5.00	3.85×10^{-16}	3	2	gradient
MGH32	BFGS (backtracking)	5.00	5.58×10^{-16}	2	2	gradient
MGH33	FFJM2 (PSB)	2.32	6.33×10^{-11}	12	5	gradient
MGH33	BFGS (backtracking)	2.32	1.42×10^{-10}	13	5	gradient
MGH34	FFJM2 (PSB)	3.07	1.55×10^{-10}	9	2	gradient
MGH34	BFGS (backtracking)	3.07	2.13×10^{-10}	9	2	gradient

With this run, FFJM2 (PSB) no longer certifies Step/Function convergence on MGH10 and MGH16, or Function convergence on MGH31: all three now stop under **Stalled** instead, meaning the regularization parameter hit its ceiling before a step could be accepted. In every case the final objective value f is essentially unchanged from the previous run (MGH10: 4.397293×10^1 ; MGH16: 4.291110×10^4 ; MGH31: 1.528639×10^0 , all matching to 6 significant figures), and the gradient norm is, if anything, smaller than before — so the iterate itself is not worse, only the exit status is: the method now keeps trying (and re-growing the regularization parameter) past the point where it previously accepted a stall as final, until it exhausts its regularization budget instead. BFGS is unaffected on all three (still Step/Function/Gradient, respectively, with essentially the same f and $\|\nabla f\|$ as before).

The boxplot below compares the performance of the two approaches. It was not regenerated for this update: its data selection already restricts to problems where *both* methods converge by the gradient criterion, which excludes MGH10, MGH16, and MGH31 on the FFJM2 side both before and after this change, so the boxplot and its Mann-Whitney test are unaffected by it.

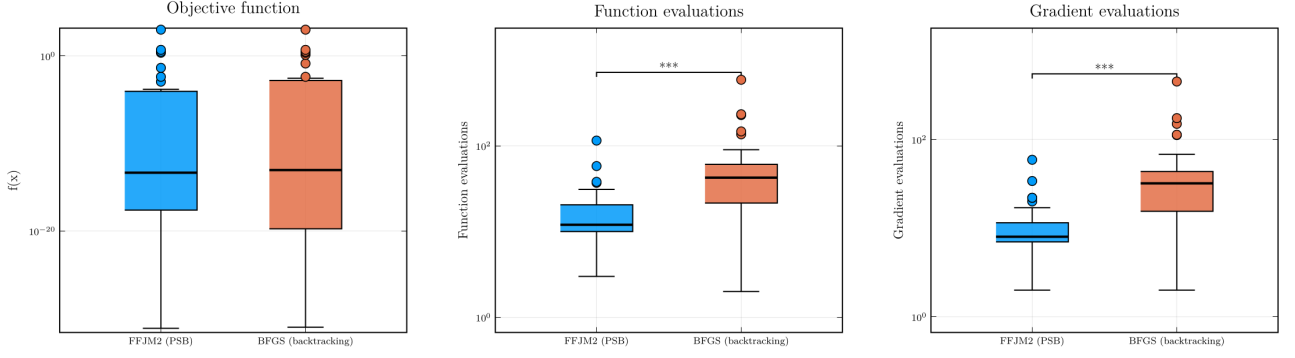


Figure 1: Comparison of FFJM2 (PSB) vs. BFGS (backtracking) on the MGH problems, restricted to runs where both methods converged by gradient norm. Final f value, function evaluations, and gradient evaluations. Stars indicate significance under the Mann-Whitney U test [3] ($p < 0.05$: *; $p < 0.01$: **; $p < 0.001$: ***).

As Figure 1 shows, the objective function values are not very different between the two methods: the p-value confirms neither achieves a significantly better result. Function evaluations and gradient evaluations, however, are different, and the p-value confirms this difference is significant. FFJM2 therefore performs better than BFGS in this respect, reaching a comparable solution at a much lower cost.

As an aside, the BFGS baseline above uses backtracking with quadratic interpolation (order 2); Table 4 checks how much that particular choice of line search matters on its own, running `Optim.BFGS` on the same 34 MGH problems with four line searches: `LineSearches.HagerZhang()` (`Optim.jl`’s default), a minimalist geometric backtracking with no interpolation (“order 1”, always restarting from a fixed step of 1), and `BackTracking` with quadratic (order 2) and cubic (order 3) interpolation. “Converged (gradient)” counts how many of the 34 problems stopped under the gradient criterion; the remaining columns are mean $\pm$ standard deviation over all 34 problems (including the ones that stopped under a different criterion).

Table 4: Comparison of line searches for `Optim.jl`’s BFGS on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Hager-Zhang	$1.29 \times 10^3 \pm 7.36 \times 10^3$	33/34	66.91 ± 120.92	107.82 ± 206.62	107.82 ± 206.62
Backtracking (order 1)	$2.49 \times 10^7 \pm 1.45 \times 10^8$	30/34	51.56 ± 93.74	89.62 ± 131.90	52.47 ± 93.75
Backtracking (order 2)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	32/34	61.59 ± 99.26	86.74 ± 134.56	62.56 ± 99.17
Backtracking (order 3)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	56.91 ± 89.47	84.91 ± 132.01	57.85 ± 89.33

Hager-Zhang converges by gradient on the most problems (33/34) but at the highest average cost, and its function/gradient evaluation counts always match exactly — it evaluates f and ∇f together at every trial point, unlike the three backtracking variants (which only need ∇f once a trial is accepted). Order-1 backtracking is the cheapest per problem on average, but also the least reliable of the four (30/34). Order-2 and order-3 sit in between on both counts. No single line search dominates the other three on every metric; the differences are modest next to the FFJM2 vs. BFGS gap of Figure 1. The mean/std of f is not a useful per-problem quality measure here — MGH problems span many orders of magnitude at their true minima, so a couple of legitimately large-residual problems (MGH16, $f \approx 4.3 \times 10^4$; MGH06, $f \approx 1.0 \times 10^3$) already dominate the $1.29 \times 10^3 \pm 7.36 \times 10^3$ shared by Hager-Zhang and orders 2–3, all three of which land on the same, correct value on MGH10 ($f \approx 44$) and everywhere else they converge. Order-1 backtracking’s much larger $2.49 \times 10^7 \pm 1.45 \times 10^8$ is a genuine failure, not scale: it stalls at x^0 on MGH10 after a single rejected trial ($f \approx 8.5 \times 10^8$, seven orders of magnitude above the correct value), the only case among the four where this happens.

A second aside: FFJM2’s quartic subproblem (Section 2, Eq. (18)/(21)) is solved internally by `Ipopt`, but the paper’s Algorithm 4.1 does not require that particular choice. Table 5 checks how the four solvers already available as `model_solver` in `ffjm2` — `Ipopt`, BFGS (backtracking, same as above),

BOBYQA, and MADS — fare when applied *directly* to $0.5\|F(x)\|^2$ on the 34 MGH problems (i.e., as if each one were, on its own, the whole optimizer, not just the subproblem solver inside FFJM2’s outer loop). Same columns as Table 4; BOBYQA and MADS are derivative-free, so their gradient-evaluation count is always 0 and “Converged (gradient)” for them instead counts $\|\nabla f\| \leq 10^{-8}$ checked *after* the solver stops (Ipopt and BFGS report their own gradient-based termination directly).

Table 5: Comparison of the solvers available for FFJM2’s internal subproblem, applied directly to $0.5\|F(x)\|^2$ on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
Ipopt	$1.88 \times 10^5 \pm 1.09 \times 10^6$	32/34	69.53 ± 103.06	424.62 ± 803.24	71.53 ± 103.06
BFGS (backtracking)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	32/34	61.59 ± 99.26	86.74 ± 134.56	62.56 ± 99.17
BOBYQA	$1.47 \times 10^{10} \pm 8.56 \times 10^{10}$	10/34	572.71 ± 699.05	572.71 ± 699.05	0.00 ± 0.00
MADS	$1.47 \times 10^{10} \pm 8.56 \times 10^{10}$	0/34	986.82 ± 594.53	986.82 ± 594.53	0.00 ± 0.00

Ipopt and BFGS are essentially tied on reliability (32/34) and land within a factor of 2 of each other on iterations and gradient evaluations, but Ipopt needs about $5\times$ more function evaluations on average (it solves a full barrier subproblem at each outer iteration). BOBYQA and MADS are far behind on both counts: with no gradient to exploit, they need an order of magnitude more evaluations and still only certify gradient convergence on 10/34 (BOBYQA) or none at all (MADS) within the 2000 function-evaluation budget. As Table 6 below shows, this gap does *not* carry over once these solvers are used the way FFJM2 actually uses them — as the resolver of the small, cheap quartic model, rather than as the whole optimizer applied to the raw residual. As in Table 4, the mean/std of f is dominated by outliers rather than typical behavior, and here they point to a genuine methodological limitation of the box these two derivative-free solvers need (BOBYQA and MADS require finite bounds; both are given $x^0 \pm \text{bound_radius}$, $\text{bound_radius} = \max(10^3, 100 \max(1, \|x^0\|))$): on MGH04, whose true minimizer is $x^* \approx (10^6, 2 \times 10^{-6})$, this box does not even contain x^* when $x^0 = (1, 1)$ gives $\text{bound_radius} = 10^3$, so both solvers stall at the box boundary ($f \approx 5.0 \times 10^{11}$, versus $\approx 8 \times 10^{-28}$ for Ipopt/BFGS on the same problem) — a box sized from $\|x^0\|$ alone cannot help when x^* is orders of magnitude farther from x^0 than x^0 is from the origin. Ipopt’s own outlier is different in kind: on MGH10 it converges (LOCALLY_SOLVED) to a KKT point with $f \approx 6.3 \times 10^6$, well above BFGS’s $f \approx 44$ on the same problem, i.e. a genuine bad local solution rather than a boundary artifact.

Table 6 repeats the comparison, but now running `ffjm2` in full (`update=:psb`) on the same 34 problems, once per `model_solver` — i.e., Ipopt/BFGS/BOBYQA solve only the small quartic model at each outer iteration, exactly as FFJM2 uses them, instead of standing in as the whole optimizer. `function_evaluations/gradient_evaluations` here count only calls to the real residual/Jacobian made by FFJM2’s outer loop (`residual(x)/jac(x)`); the internal solver’s own work on the quartic surrogate is tracked separately by `ffjm2` (fields `model_solves/model_iterations/model_solve_time_seconds`, not shown) and never mixed into these counts, so this is already an apples-to-apples measure of external cost regardless of `model_solver`. MADS is left out here: even at a much smaller `model_multistart` than the default (10 instead of 1000), a single easy problem (MGH01) took ≈ 69 s in MADS alone against < 0.1 s for BFGS/BOBYQA, making the default multi-start impractical across all 34 problems for this table.

Table 6: FFJM2 (PSB), varying only `model_solver`, on the 34 MGH problems.

Method	f	Converged (gradient)	Iterations	Func. evals	Grad. evals
FFJM2 (PSB, Ipopt)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	15.12 ± 21.90	30.21 ± 49.59	16.12 ± 21.90
FFJM2 (PSB, BFGS)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	31/34	15.26 ± 21.99	30.26 ± 48.63	16.26 ± 21.99
FFJM2 (PSB, BOBYQA)	$1.29 \times 10^3 \pm 7.36 \times 10^3$	30/34	39.74 ± 108.86	57.68 ± 127.48	40.74 ± 108.86

The gap seen in Table 5 almost disappears here: Ipopt and BFGS as `model_solver` are essentially indistinguishable (31/34, within 1% of each other on every column), and even BOBYQA — which needed an order of magnitude more evaluations and converged on only 10/34 when applied directly to the raw residual — falls behind by only one problem (30/34) and roughly $2.6\times$ more outer iterations on average, not $10\times$. All three land on the same f statistics, again driven by the legitimately large-

residual MGH16/MGH06 rather than any failure. This is exactly the effect anticipated when this experiment was designed (`teste.jl`): FFJM2’s own outer loop — the ratio test that rejects a bad step and grows σ , and the full Hessian reset that follows a bad ratio (see the discussion of σ in Section 1.1) — compensates for a mediocre subproblem solution, so a weaker `model_solver` costs a few extra outer iterations rather than failing outright. FFJM2 still defaults to Ipopt — tied with BFGS on reliability (31/34) and within a step or two of it on every problem where they differ (worst case MGH10, 121 outer iterations against BFGS’s 120) — but the choice matters far less than Table 5 alone would suggest.

2 Coefficient estimation problem

The Saint-Venant equations [?, ?, ?, ?] describe the flow of fluids in natural channels. The state variables that describe a natural channel vary with respect to $x \in [x_{min}, x_{max}]$ (position, usually measured in meters) and $t \in [t_{min}, t_{max}]$ (time, usually measured in seconds). We define:

- $z(x, t)$: surface elevation
- $h(x, t) = z(x, t) - z_b(x)$: river depth at (x, t)
- $A(x, t)$: cross-sectional wetted area
- $Q(x, t)$: flow rate
- $P(x, t)$: wetted perimeter
- $R(x, t) = A(x, t)/P(x, t)$: hydraulic radius
- g : acceleration of gravity (approximately $9.81m/s^2$)

The Saint-Venant equations consist of two main components: Equation (1) describes mass conservation and equation (2) represents conservation of the linear momentum.

$$\frac{\partial A}{\partial t} + \frac{\partial Q}{\partial x} = 0 \quad (1)$$

and

$$\frac{\partial Q}{\partial t} + \frac{\partial}{\partial x} \left(\frac{Q^2}{A} \right) + gA \frac{\partial z}{\partial x} + \frac{n_g^2 Q |Q|}{AR^{4/3}} = 0. \quad (2)$$

The Manning (also called Yen-Manning [?]) coefficient n_g represents roughness [?]. Typically, given the inlet discharge $Q(x_{min}, t)$ for $t \in [t_{min}, t_{max}]$, initial conditions $A(x, t_{min})$ and $Q(x, t_{min})$, and the coefficient n_g , numerical methods compute the values of $Q(x, t)$ and $A(x, t)$ at a set of discrete “grid points” (x_ν, t_ν) . Unfortunately, the Yen-Manning coefficient n_g is often unknown for natural channels and can vary with position x and even time t . For more details, see [?].

This research assumes that the Manning coefficient n_g varies with position x , and that a set of elevation observations $z_{obs}(x_\nu, t_\nu)$ are available at n_{obs} distinct locations and times, indexed by $\nu = 1, \dots, n_{obs}$. Consequently, the spatial distribution of $n_g(x)$, discretized over a suitable grid, becomes an unknown component of our problem. Furthermore, to ensure physically plausible results, minimal variation will be required for a function $n_g(x)$ that minimizes spatial oscillations while fitting the solution of the Saint-Venant equations to the observations.

Following the notation of Section 3 — reusing x for this new, unrelated role as the vector of unknowns, distinct from the spatial coordinate used above — collect the n discretized values of n_g into a parameter vector $x \in \mathbb{R}^n$. For $i = 1, \dots, m$ (with $m = n_{obs}$), let $z_i(x)$ denote the surface elevation the Saint-Venant solution predicts, for parameter vector x , at the i -th observation’s location and time (x_i, t_i) , and let $z_{obs,i} = z_{obs}(x_i, t_i)$ be the corresponding observation. Exactly as in Section 3, define the residual components $f_i(x) = z_i(x) - z_{obs,i}$, collect them into $F(x) = (f_1(x), \dots, f_m(x))^T$, and minimize

$$f(x) = \frac{1}{2} \|F(x)\|_2^2 = \frac{1}{2} \sum_{i=1}^m f_i(x)^2.$$

Calibrating n_g against the observations is thus the nonlinear least-squares problem of finding x that minimizes $f(x)$ (ideally $F(x) = 0$, though the real data need not be exactly consistent with the model).

To calibrate x against this residual, four benchmark scenarios compare the same three solvers used throughout this report — BFGS (backtracking, order 2), BOBYQA, and **ffjm2** — applied directly to $F(x)$ above, in the same way as Table 5: each solver stands in as the whole optimizer, rather than as **ffjm2**’s internal subproblem solver. Two scenarios are *twin* experiments, where the observations $z_{obs,i}$ are themselves generated by the model at a known x^* , so the true minimizer is known exactly; the other two use the real observed data, where the true minimizer (if it exists) is unknown. BFGS and BOBYQA minimize the boxed, penalized sum of squares $\|F(x)\|_2^2 + \text{penalty}(x)$ (box $[0, 0.5]^n$, penalty weight 10^6); **ffjm2** minimizes the unpenalized $f(x) = \frac{1}{2}\|F(x)\|_2^2$ directly, so its f column below is, on runs where the box constraint is inactive, exactly half of the other two methods’ — e.g. real data, $n = 3$, below, where all three reach the same minimizer up to rounding: $f \approx 1.0085$ for BFGS/BOBYQA versus $f \approx 0.5043$ for **ffjm2**. RMSD is recomputed uniformly from the raw (unpenalized) residual, $\text{RMSD}(x) = \|F(x)\|_2/\sqrt{m}$, at each method’s minimizer, so it is directly comparable across all rows of every table below.

Table 7: Twin experiment, $n = 2$: $x^* = (0.20, 0.15)$, $x^0 = (0.09, 0.09)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	6.95×10^{-9}	1.63×10^{-14}	1.19×10^{-5}	17	9	416.1	GradientNorm
BOBYQA	1.56×10^{-9}	8.20×10^{-16}	2.91×10^{-6}	43	0	456.2	XTOL_REACHED
ffjm2	3.04×10^{-15}	1.57×10^{-27}	1.36×10^{-13}	11	6	200.5	gradient_converged

All three methods converge to essentially the exact reference $x^* = (0.20, 0.15)$; **ffjm2** is the cheapest of the three by every cost measure (fewest evaluations, least time) despite also reaching the tightest gradient norm and RMSD.

Table 8: Twin experiment, $n = 10$: decreasing profile x^* from 0.12 to 0.07, $x^0 = (0.09, \dots, 0.09)$. (Increasing profiles over the same range were tested manually and diverge before $t_{\max} = 31$; only decreasing profiles are stable.)

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	1.74×10^{-6}	1.03×10^{-9}	1.32×10^{-4}	205	168	9959.2	GradientNorm
BOBYQA	6.80×10^{-5}	1.56×10^{-6}	3.27×10^{-4}	1000	0	12037.3	MAXEVAL_REACHED
ffjm2	5.02×10^{-13}	4.26×10^{-23}	1.89×10^{-11}	29	17	942.7	gradient_converged

At $n = 10$ the gap between **ffjm2** and the other two widens sharply: it reaches a gradient norm roughly 7 orders of magnitude smaller than BFGS or BOBYQA, using about 7–35 $\times$ fewer function evaluations and 10–13 $\times$ less wall-clock time. BOBYQA exhausts its 1000-evaluation budget without reaching the gradient criterion (MAXEVAL_REACHED, converged=false).

Table 9: Real data, $n = 3$: $x^0 = (0.09, 0.09, 0.09)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	5.46×10^{-2}	1.0085	3.85×10^{-6}	25	15	738.5	GradientNorm
BOBYQA	5.46×10^{-2}	1.0085	1.06×10^{-4}	116	0	1432.3	XTOL_REACHED
ffjm2	5.46×10^{-2}	0.5043	6.41×10^{-9}	26	12	553.6	gradient_converged

On real data, all three methods agree closely on the minimizer itself: RMSD matches to three significant figures across the board ($\approx 5.46 \times 10^{-2}$), and f matches once **ffjm2**’s factor of 2 (discussed above) is accounted for. **ffjm2** again reaches by far the smallest gradient norm — roughly three orders of magnitude below BFGS — at an evaluation count and wall-clock time comparable to BFGS.

Table 10: Real data, $n = 10$: $x^0 = (0.09, \dots, 0.09)$.

Method	RMSD	f	$\ \nabla f\ $	Func. evals	Grad. evals	Time (s)	Status
BFGS (backtracking)	4.81×10^{-2}	0.7822	1.19×10^6	171	35	3654.2	<code>alpha_too_small</code>
BOBYQA	4.64×10^{-2}	0.7285	1.11	840	0	10297.8	SUCCESS
ffjm2	not available in this run						

This is the hardest of the four scenarios. BFGS’s backtracking line search stalls (`alpha_too_small`, `converged=false`) with a gradient norm still at $\sim 1.19 \times 10^6$, many orders of magnitude away from a stationary point, despite needing fewer evaluations and less time than BOBYQA. BOBYQA reports SUCCESS and reaches the lowest RMSD (4.64×10^{-2}), but needs 840 evaluations and ≈ 2.9 h of wall-clock time to get there. `ffjm2` was not run (or did not complete) for this scenario; that row is left for a future update once `comparar_solvers_real_dim10` has been re-run with it included.

References

- [1] J. E. Dennis, Jr. and H. Wolkowicz, *Sizing and least-change secant methods*, SIAM J. Numer. Anal., 30 (1993), pp. 1291–1314.
- [2] L. Lukšan and E. Spedicato, *Variable metric methods for unconstrained optimization and nonlinear least squares*, J. Comput. Appl. Math., 124 (2000), pp. 61–95.
- [3] H. B. Mann and D. R. Whitney, *On a test of whether one of two random variables is stochastically larger than the other*, Ann. Math. Statist., 18 (1947), pp. 50–60.
- [4] P. K. Mogensen and A. N. Riseth, *Optim: A mathematical optimization package for Julia*, J. Open Source Software, 3 (2018), p. 615.
- [5] J. J. Moré, B. S. Garbow, and K. E. Hillstom, *Testing unconstrained optimization software*, ACM Trans. Math. Software, 7 (1981), pp. 17–41.
- [6] M. J. D. Powell, *A new algorithm for unconstrained optimization*, in Nonlinear Programming, J. B. Rosen, O. L. Mangasarian, and K. Ritter, eds., Academic Press, New York, 1970, pp. 31–65.